



FlashArray Zero Touch Provisioning

Generated on: 28 May 2026

Contents

- FlashArray Zero Touch Provisioning 3
 - Availability 3
 - The Remote Deployment Process 4
 - Physical Installation of the FlashArray 4
 - Setting up DHCP Service 4
 - Querying the FlashArray 6
 - Initializing the FlashArray 8
 - Verification 14
 - Limitations 15

FlashArray Zero Touch Provisioning

Currently, the initial deployment of a FlashArray™ requires a person to be physically present in the Data Center. This is inconvenient for some customers; particularly those that deploy a large number of FlashArrays.

To address this concern, Pure Storage® introduced a new DHCP boot feature, in which the management ports ct0.eth0 & ct1.eth0 on the FlashArray request IP addresses from a DHCP server when the array is first powered on. A REST API endpoint is also added on ct1.eth0, so that after the FlashArray powers on for the first time, users can connect to it remotely via the REST API, and initialize the array. This process can be performed remotely and eliminates the need for a direct connection to the FlashArray via the console port for the initial setup.

Note:

Both management interfaces must be configured on both controllers and both arrays with enabled and active links. The management interfaces are as follows:

- FlashArray//XR4 and FlashArray XR5 - ct0.eth4, ct0.eth5, ct1.eth4, ct1.eth5
- All other FlashArray Models - ct0.eth0, ct0.eth1, ct1.eth0, ct1.eth1

When the FlashArray first powers up in DHCP mode, no authentication is required to connect to the FlashArray via the REST API endpoint. Users can connect to the endpoint via the IP address assigned by the DHCP server. During the initialization process, the DHCP assigned IP addresses are replaced with static IP addresses. When the initialization process is complete, the FlashArray returns to its normal operating mode, the DHCP feature is disabled, and the FlashArray no longer allows remote REST API connections without authentication.

Availability

- The FlashArray remote deployment feature is available in Purity versions 5.2.4 and newer.
- Zero Touch Provisioning will soon be available for FlashArray //XR5 and //XLR5 models that ship with Factory Image 1.2.0 (after late Feb 2026).
 - You can determine the factory image version with `pureversion -a`

```
Product Version: 1.2.0
Middleware NDU Compatibility: 1.2.0
Operating Environment: factoryimage-fa
```

The Remote Deployment Process

Following are the steps to remotely deploy a FlashArray:

Physical Installation of the FlashArray

A new FlashArray pre-installed with Purity version 5.2.4 or above should be shipped to the customer's Data Center. When the array is unpacked, if the serial number of the FlashArray and the MAC addresses of the management ports are printed on the array or on a document accompanying the array, this information should be saved. The FlashArray should then be racked and cabled up.

Ensure that DHCP service is running before powering-on the FlashArray.

Setting up DHCP Service

A DHCP server must be running when the FlashArray is initially powered on so that when the management ports on the FlashArray broadcast discover packets to request IP addresses. The DHCP server responds and assigns addresses from its IP pool to the management ports on the array.

Optional step: For convenience, the DHCP server can be configured to assign specific IP addresses to the management ports on the FlashArray, based on the MAC address information that was found when unpacking the FlashArray. The advantage of using MAC-based IP address reservations in a DHCP server is that you don't have to search through the whole range of IP addresses in the DHCP server's IP pool to find the IP address of the FlashArray.

The following screenshot shows an example of how to do MAC-based reservation of an IP address on a Windows 2016 DHCP server:

New Reservation ? X

Provide information for a reserved client.

Reservation name:

IP address:

MAC address:

Description:

Supported types

☐ Both

☒ DHCP

☐ BOOTP

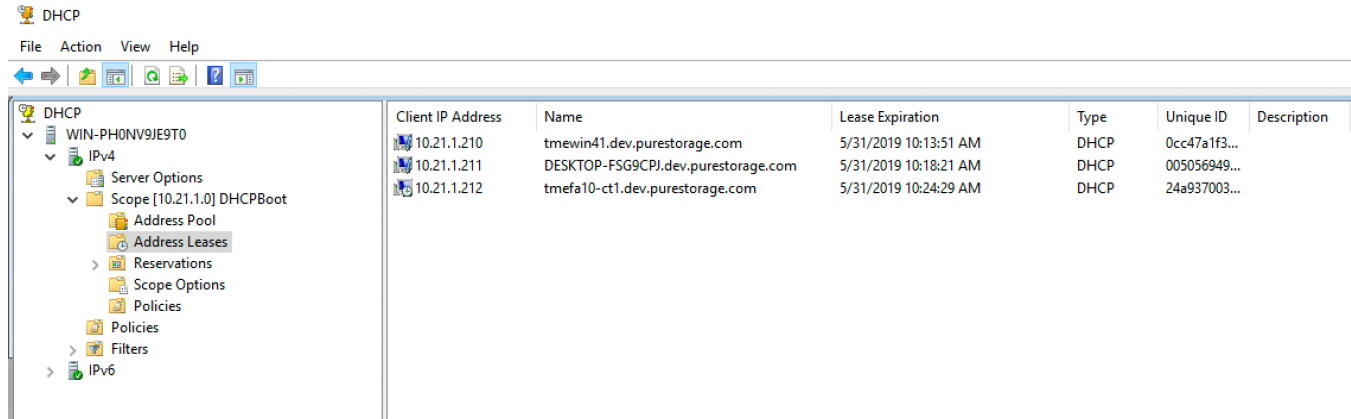
Add **Close**

Note that the above screen may vary based on the DHCP server version.

After powering on the FlashArray, query the DHCP server and find the IP addresses assigned to ct0.eth0 & ct1.eth0. If the DHCP server was configured to assign specific IP addresses to the FlashArray based on MAC addresses, it is still a good idea to verify that those IP addresses are assigned as expected. When searching for IP addresses, the FlashArray will identify itself to the DHCP server using broadcasted DHCP discover messages.

In the following example, a Windows 2016 DHCP Server is queried to find the IP address of controller ct1

on the "tmefa10" FlashArray:



The screenshot shows a DHCP console window with a tree view on the left and a table of leases on the right. The tree view shows a hierarchy: DHCP > WIN-PH0NV9JE9T0 > IPv4 > Scope [10.21.1.0] DHCPBoot > Address Leases. The table on the right lists three leases with columns: Client IP Address, Name, Lease Expiration, Type, Unique ID, and Description.

Client IP Address	Name	Lease Expiration	Type	Unique ID	Description
10.21.1.210	tmewin41.dev.purestorage.com	5/31/2019 10:13:51 AM	DHCP	0cc47a1f3...	
10.21.1.211	DESKTOP-FSG9CP1.dev.purestorage.com	5/31/2019 10:18:21 AM	DHCP	005056949...	
10.21.1.212	tmefa10-ct1.dev.purestorage.com	5/31/2019 10:24:29 AM	DHCP	24a937003...	

Querying the FlashArray

After finding the IP addresses of ct0.eth0 and ct1.eth0, it's a good idea to ping these ports to confirm the IP addresses. Optionally, for further confirmation, you can also ssh into ct0.eth0 and/or ct1.eth0 as "pureuser". Please note that you won't be able to execute most of the commands after logging in, since the FlashArray is not initialized at this point.

After confirming the IP addresses, send a query to the FlashArray via a **GET /array-initial-config** request to the REST API endpoint at port 8081 on port ct1.eth0. No authentication is needed to connect to the FlashArray at this step.

Note that there's no REST API endpoint on port ct0.eth0, so the FlashArray will not respond if a similar GET request is sent to port ct0.eth0.

Following is an example of a **GET /array-initial-config** Request to a FlashArray, where the IP address of the management port ct1.eth0 is 10.21.1.212:

GET REQUEST URL = <http://10.21.1.212:8081/array-initial-config>

If communication with the FlashArray is successful, the HTTP response status code should be "200 OK", and the response to the GET request should be similar to the following:

```
{
  "status": "uninitialized",
  "array_id": "",
  "ct0.eth0": {
    "netmask": "255.255.255.0",
    "address": "10.21.1.171",
    "gateway": "10.21.1.1",
```

```

    "mac_address": "24:a9:37:00:34:33"
  },
  "smtp": {
    "relay_host": "",
    "sender_domain": ""
  },
  "vir0": {
    "netmask": "",
    "mac_address": "",
    "gateway": "",
    "address": ""
  },
  "alert_emails": [],
  "ct1.eth0": {
    "netmask": "255.255.255.0",
    "mac_address": "24:a9:37:00:34:39",
    "gateway": "10.21.1.1",
    "address": "10.21.1.212"
  },
  "ntp_servers": [],
  "version": "5.2.4",
  "eula_acceptance": {
    "accepted_by": {
      "organization": "",
      "full_name": "",
      "job_title": ""
    },
    "accepted": false
  },
  "progress": 0,
  "dns": {
    "nameservers": [],
    "domain": ""
  },
  "serial_number": "PCHFL17320104",
  "timezone": "America/Los_Angeles",
  "os": "Purity//FA",
  "array_name": "tmeffa10"
}

```

Check the above information to verify that the FlashArray is in an uninitialized state, and confirm that the serial number & the MAC addresses match the information noted down when the system was unpacked.

The next step is to initialize the FlashArray via the REST API endpoint, and assign static IP addresses to the array.

Initializing the FlashArray



Note: When using the ZTP tool to provision RC4 arrays that use ETH4/5 as their management ports, the config.json file that is used must be modified to specify ct0.eth4, ct1.eth4, and vir4 as the interfaces that are being given IPs. This is instead of the original eth0 and vir0 ports that are typically used.

If this is not done, an error with status code 400 will show 'invalid key vir0 in array configuration'. An example is shown [here](#).

Following is the information required to initialize the FlashArray. Please note that except for the fields marked as "optional", the rest of the information must be provided in order to initialize the FlashArray:

Information Required to Initialize the FlashArray			
FlashArray Name			
vir0	IP Address	Netmask	Gateway
ct0.eth0	IP Address	Netmask	Gateway
ct1.eth0	IP Address	Netmask	Gateway
DNS Domain (optional)	Name Servers		
NTP Servers			
Time Zone			

Information Required to Initialize the FlashArray			
SMTP Relay Host (optional)	Sender Domain	Alert Email Addresses (optional)	
Agreement to Accept the EULA	Full Name	Job Title	Name of Organization

The above information has to be put into the JSON format, as shown in the example below, and sent to the FlashArray in a PATCH request over the REST API. Do not use the format shown in the GET command above, as it contains additional fields for which the FlashArray does not accept user input, such as the array serial number.

Copy the text in the example below to create the JSON configuration. Replace the sample information with the information specific to your environment. Then use the following PATCH Request to send this configuration to the FlashArray via the REST API endpoint at ct1.eth0.

PATCH Request URL = <http://10.211.212:8081/array-initial-config>

Where: 10.211.212 is the IP address of ct1.eth0 on the FlashArray

Following is an example of the information that should be sent in the body of the PATCH Request to initialize the FlashArray (for more information on JSON, check out <http://www.json.org/>):

```
{
  "array_name": "tmeffa10",
  "ct0.eth0":
  {
    "address": "10.21.1.183",
    "netmask": "255.255.255.0",
    "gateway": "10.21.1.1"
  },
  "ct1.eth0":
  {
    "address": "10.21.1.184",
    "netmask": "255.255.255.0",
    "gateway": "10.21.1.1"
  },
  "vir0":
```

```

{
  "address": "10.21.1.182",
  "netmask": "255.255.255.0",
  "gateway": "10.21.1.1"
},
"dns":
{
  "domain": "dev.purestorage.com",
  "nameservers":
  [
    "10.14.240.240"
  ]
},
"ntp_servers":
[
  "time1.purestorage.com"
],
"timezone": "America/Los_Angeles",
"smtp":
{
  "relay_host": "blackhole-smtp2.dev.purestorage.com",
  "sender_domain": "purestorage.com"
},
"alert_emails":
[
  "amin@purestorage.com"
],
"eula_acceptance":
{
  "accepted": true,
  "accepted_by":
  {
    "organization": "Pure Storage",
    "full_name": "Syed Amin",
    "job_title": "TME"
  }
}
}

```

Please note in the EULA (End User License Agreement) section at the end of the JSON file. By entering the following information into the JSON file, you, as the customer, are agreeing to the terms and conditions in the EULA:

```

"eula_acceptance":
{
    "accepted": true,
    "accepted_by":
    {
        "organization": "ABC Corporation",
        "full_name": "John Smith",
        "job_title": "System Administrator"
    }
}

```

If the PATCH Request is successful, and there are no error messages, the FlashArray will send a response similar to the following:

```

{
    "status": "initializing",
    "array_id": "",
    "ct0.eth0":
    {
        "netmask": "255.255.255.0",
        "address": "10.21.1.183",
        "gateway": "10.21.1.1",
        "mac_address": "24:a9:37:00:34:33"
    },
    "smtp":
    {
        "relay_host": "blackhole-smtp2.dev.purestorage.com",
        "sender_domain": "purestorage.com"
    },
    "vir0":
    {
        "netmask": "255.255.255.0",
        "address": "10.21.1.182",
        "gateway": "10.21.1.1",
        "mac_address": ""
    }
}

```

```

    },
    "alert_emails":
    [
        "amin@purestorage.com"
    ],
    "ct1.eth0":
    {
        "netmask": "255.255.255.0",
        "gateway": "10.21.1.1",
        "address": "10.21.1.184"
    },
    "ntp_servers":
    [
        "time1.purestorage.com"
    ],
    "version": "5.2.4",
    "eula_acceptance":
    {
        "accepted_by":
        {
            "organization": "Pure Storage",
            "full_name": "Syed Amin",
            "job_title": "TME"
        },
        "accepted": true
    },
    "progress": 0.01,
    "dns":
    {
        "nameservers":
        [
            "10.14.240.240"
        ],
        "domain": "dev.purestorage.com"
    },
    "timezone": "America/Los_Angeles",
    "serial_number": "PCHFL17320104",
    "os": "Purity//FA",
    "array_name": "tme_fa10"
}

```

Once the FlashArray has been initialized successfully, use the static IP addresses that were assigned to the array during initialization in order to communicate with the FlashArray. The DHCP assigned IP

addresses will no longer be valid. Also, the "array-initial-config" request will no longer be valid. Therefore, authentication is required to communicate with the FlashArray via the REST API endpoint.

Configuration Examples

Typical JSON config example:

```
{
  "array_name": "slc-314-fa01",
  "ct0.eth0":
  {
    "address": "10.235.118.73",
    "netmask": "255.255.252.0",
    "gateway": "10.235.116.1"
  },
  "ct1.eth0":
  {
    "address": "10.235.118.74",
    "netmask": "255.255.252.0",
    "gateway": "10.235.116.1"
  },
  "vir0":
  {
    "address": "10.235.118.72",
    "netmask": "255.255.252.0",
    "gateway": "10.235.116.1"
  },
}
```

Modified config example with ETH4/VIR4

```
{
  "array_name": "slc-314-fa01",
  "ct0.eth4":
  {
    "address": "10.235.118.73",
    "netmask": "255.255.252.0",
    "gateway": "10.235.116.1"
  },
  "ct1.eth4":
  {
    "address": "10.235.118.74",
    "netmask": "255.255.252.0",
    "gateway": "10.235.116.1"
  },
}
```

```

},
"vir4":
{
"address": "10.235.118.72",
"netmask": "255.255.252.0",
"gateway": "10.235.116.1"
},
{

```

Verification

The final step is to query the FlashArray and verify that it has been initialized successfully. Since authentication is now required to communicate with the FlashArray, use an API token to create a secure REST API session:

The API token for any user can be copied from the FlashArray GUI by navigating to Settings > Users > user_name > Show API Token, as shown below:

The screenshot shows the Pure Storage FlashArray GUI. The left sidebar contains navigation links: Dashboard, Storage, Analysis (Performance, Capacity, Replication), Health, and Settings. The main content area is titled 'Settings' and has tabs for System, Network, Users, and Software. The 'Users' tab is selected, displaying a table of users. The table has columns: Name, Role, Type, Public Key, API Token, and Lockout Remaining. The 'pureuser' row is highlighted, and a context menu is open over it, showing options: Edit User..., Show API Token..., Recreate API Token..., and Remove API Token....

Name	Role	Type	Public Key	API Token	Lockout Remaining
opsadmin	ops_admin	local			-
pureuser	array_admin	local		****	-
readadmin	readonly	local			
root		local		****	
storeadmin	storage_admin	local		****	

After the API token has been copied, create a REST API session by sending the following POST request and payload:

POST Request URL = <https://10.21.1.184/api/1.16/auth/session>

Where: "10.21.1.184" is the IP address of the management port on the FlashArray

Include the following payload in the body of the POST Request:

```
{
  "api_token": "0adbaff8-324b-7ed5-4d64-45e58f7527de"
}
```

Where: "0adbaff8-324b-7ed5-4d64-45e58f7527de" is the API token for the FlashArray

If the Request is successful, you should get a response similar to the following:

```
{
  "username": "pureuser"
}
```

Next, send a simple GET request such as the following to the FlashArray:

GET Request URL = <https://10.21.1.184/api/1.16/array>

You should get a response similar to the following:

```
{
  "revision": "201904291840+fd767a8",
  "version": "5.2.4",
  "array_name": "tmeffa10",
  "id": "3db6c0e3-b79b-4943-bbee-29945d939050"
}
```

The above response displays basic information about the FlashArray. While the REST API session is established, you can issue other requests to the FlashArray, e.g. the **Get: <https://10.21.1.184/api/1.16/network/ct0.eth0>** request, which will return network information on the ct0.eth0 management port on the array.

Limitations

- Currently, we only support IPv4 for DHCP boot.

© 2015-2026 Pure Storage® ("Pure"), Portworx® and associated its trademarks can be found here as and its virtual patent marking program can be found here. Third party names may be trademarks of their respective owners. The Pure Storage products and programs described in this documentation are distributed under a license agreement restricting the use, copying, distribution, and decompilation/reverse engineering of the products. No part of this documentation may be reproduced in any form by any means without prior written authorization from Pure Storage, Inc. and its licensors, if any. Pure Storage may make improvements and/or changes in the Pure Storage products and/or the programs described in this documentation at any time without notice. This documentation is provided "as is" and all express or implied conditions, representations and warranties, including any implied warranty of merchantability, fitness for a particular purpose, or non-infringement, are disclaimed, except to the extent that such disclaimers are held to be legally invalid. Pure shall not be liable for incidental or consequential damages in connection with the furnishing, performance, or use of this documentation. The information contained in this documentation is subject to change without notice.
